Web Security & Vulnerability Testing (VAPT Basics)

Week 6 Assignment Report

Name:- Kritika Rai

Date: 14 October 2026

Table of Contents

1. Introduction to VAPT

2. Objective

3. Scope & Limitations

4. Tools Used

5. Lab Environment Setup

6. Testing Process

    6.1 SQL Injection

    6.2 Reflected Cross-Site Scripting (XSS)

    6.3 Stored Cross-Site Scripting (XSS)

    6.4 Automated Scanning with OWASP ZAP

7. Findings & Risk Levels

8. Mitigation Methods

9. Conclusion

10. References

1. Introduction to VAPT

VAPT stands for Vulnerability Assessment and Penetration Testing. In simple words, it means checking a website or application for security weaknesses, and then actually trying to exploit those weaknesses to see how bad the impact can be. A Vulnerability Assessment is mostly about scanning and finding known issues, while Penetration Testing is about actually trying to break in using those issues, like a real attacker would.

For this assignment, I practiced VAPT on DVWA (Damn Vulnerable Web Application). DVWA is a web app that is made on purpose to be insecure, so that students like us can learn about real vulnerabilities in a safe, legal environment instead of testing on a real website.

I mainly focused on two vulnerabilities from the OWASP Top 10 list: SQL Injection and Cross-Site Scripting (XSS). I also ran an automated scan using OWASP ZAP to check for other common issues like broken authentication and security misconfiguration.

2. Objective

- Learn about OWASP Top 10 vulnerabilities like SQL Injection, XSS, Broken Authentication and Security Misconfiguration.
- Set up DVWA in a safe lab environment using Docker.
- Learn how to use Burp Suite and OWASP ZAP.
- Actually perform SQL Injection and XSS (both reflected and stored) on DVWA.
- Run a scan using OWASP ZAP and check the results.
- Understand how these vulnerabilities can be fixed.

3. Scope & Limitations

- All testing was carried out on a local DVWA instance running in a Docker container on my own machine — not against any live or production system.
- The DVWA security level was set to Low for every test, since this level has no input validation at all and is meant for beginners to clearly see how each vulnerability works.
- Manual exploitation in this assignment was limited to SQL Injection and Cross-Site Scripting (Reflected and Stored), as required by the task. Broken Authentication and Security Misconfiguration were not manually exploited (e.g., no DVWA Brute Force module testing was performed) — they are covered here through the OWASP ZAP automated scan results and through conceptual study, not hands-on attack.
- This exercise is for educational purposes only and was conducted entirely within a private, isolated lab environment.

4. Tools Used

4.1 Burp Suite (Community Edition)

Burp Suite is a tool that sits in between the browser and the website as a proxy. This means every request that goes from the browser to the server, and every response that comes back, passes through Burp first. I used it to capture and look at the actual requests when doing SQL Injection and Stored XSS, so I could see exactly what data was being sent.

4.2 OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is a free tool used for testing the security of web apps. It can scan a website automatically and tell you what vulnerabilities it finds. I used ZAP to run an automated scan on DVWA to check for issues other than SQLi and XSS, like missing security headers.

4.3 DVWA (Damn Vulnerable Web Application)

DVWA is the target application used for this whole exercise. It has separate sections for different vulnerabilities such as SQL Injection, XSS, Command Injection, File Upload, CSRF, etc. It also has 4 security levels (Low, Medium, High, Impossible). I used the Low security level, since at this level there is no protection at all, so it's easier to understand how the vulnerability actually works.

4.4 Docker

Instead of installing XAMPP and setting up DVWA manually, I used Docker to run DVWA. Docker basically runs DVWA inside a container which already has everything it needs (PHP, MySQL, etc.), so the setup was much faster and easier.

5. Lab Environment Setup

First, I pulled and ran the DVWA image using Docker with this command, which maps port 8080 on my machine to port 80 inside the container:

```
sudo docker run -d -p 8080:80 vulnerables/web-dvwa
```

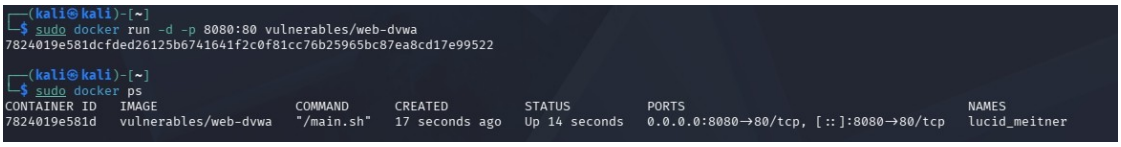


Fig 5.1 - DVWA container up and running (checked using docker ps).

After the container started, I opened it in the browser and set up the database from the Setup page, then logged in with the default login (admin / password):

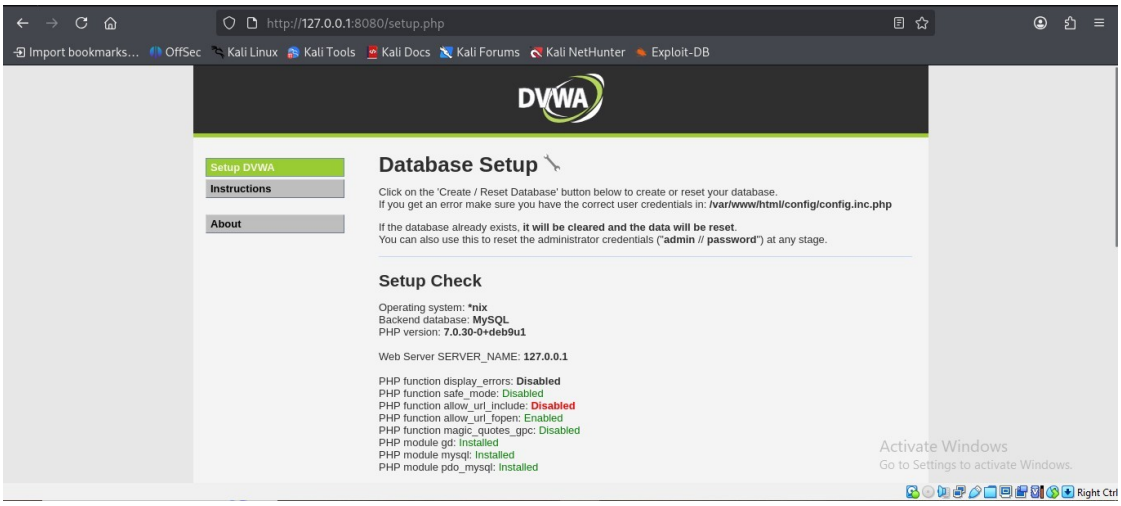


Fig 5.2 - DVWA database setup page, showing PHP/MySQL are installed correctly.

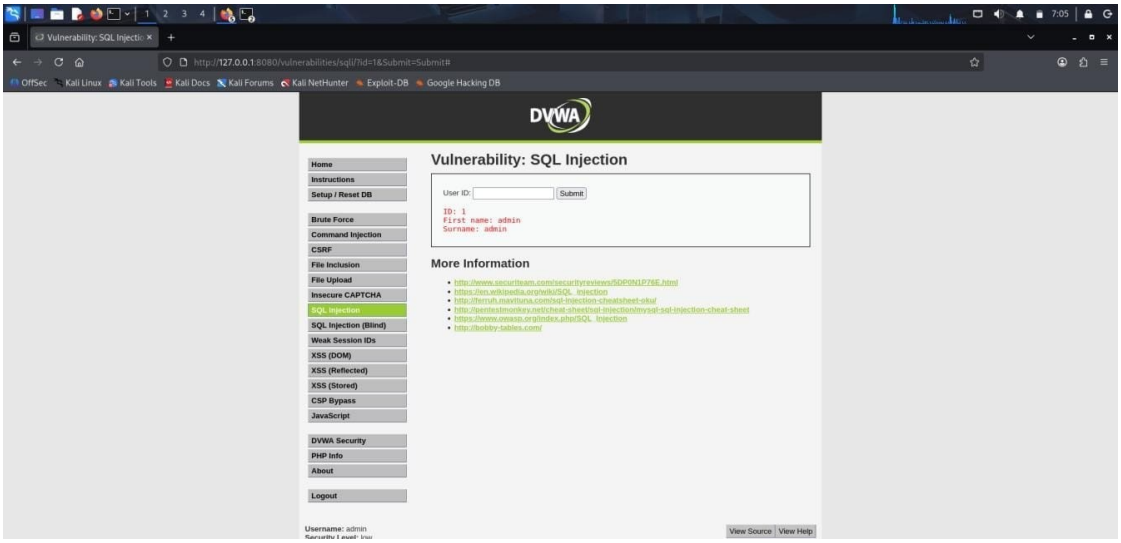


Fig 5.3 - Logged into DVWA as admin.

Next, I went to DVWA Security settings and set the security level to Low, so that none of the vulnerabilities are blocked:

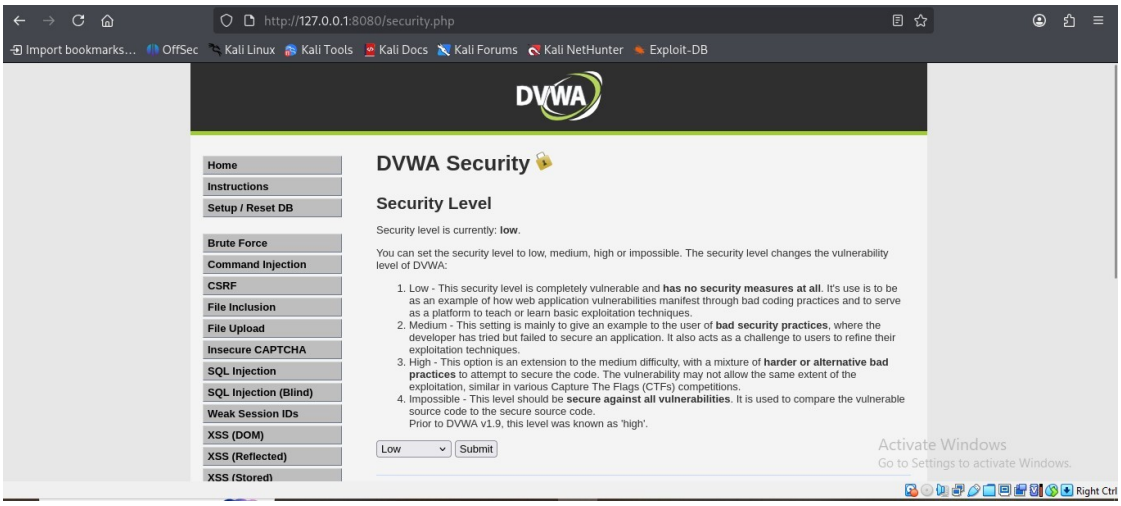


Fig 5.4 - DVWA security level set to Low.

6. Testing Process

6.1 SQL Injection

Tested on: DVWA → SQL Injection page.

First, I tried the normal way. I entered User ID as 1, and it just showed me the details of one user (admin), which is the expected behaviour.

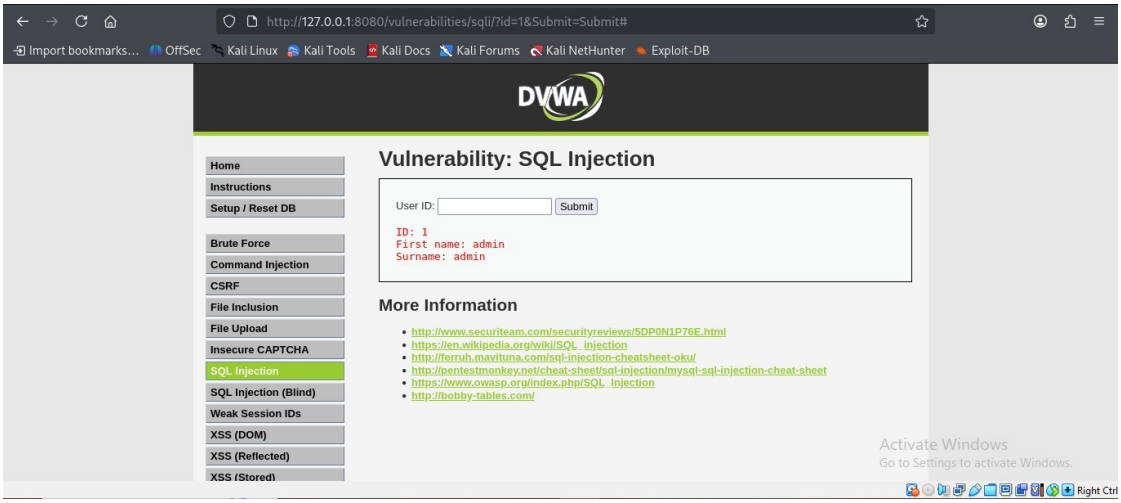


Fig 6.1 - Normal input (User ID = 1) returns one record.

Then I entered this payload in the same field instead of a normal ID:

```
1' OR '1'='1
```

This payload makes the condition in the SQL query always true, so instead of giving just one user, it returned every single user stored in the database. This happens because the app directly puts whatever we type into the SQL query, without checking it first.



Fig 6.2 - Payload returns all users (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) instead of just one.

I also captured this same request in Burp Suite to see the raw request and response:

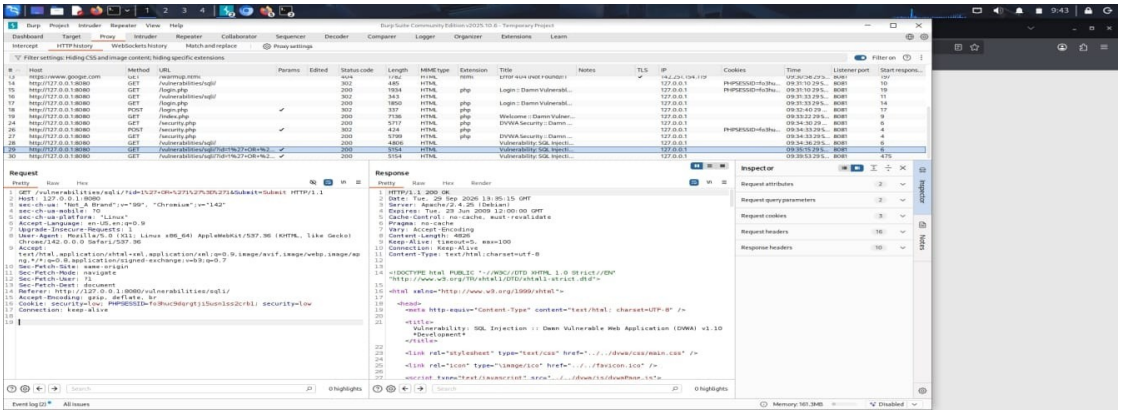


Fig 6.3 - Burp Suite capture of the SQL Injection request and response.

Conclusion: This page is vulnerable to SQL Injection because it doesn't validate or sanitize the input before using it in the query.

6.2 Reflected Cross-Site Scripting (XSS)

Tested on: DVWA → XSS (Reflected) page.

Normally, if I type a name like 'kritika' in the box, it just prints it back as 'Hello kritika'.

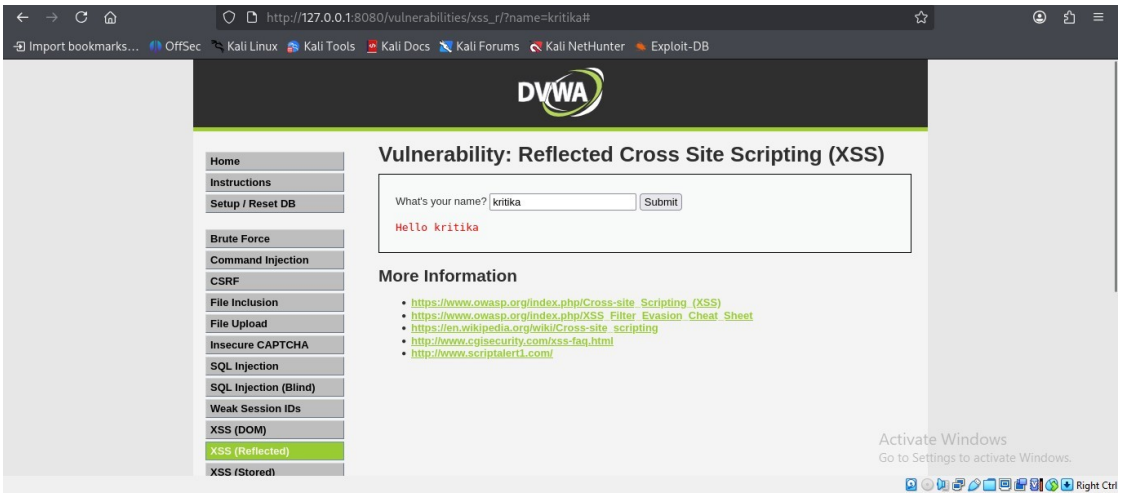


Fig 6.4 - Normal input is just printed back as it is.

But if I enter a script instead of a name, like this:

```
<script>alert('XSS')</script>
```

the page doesn't just print it as text, it actually runs it as code, and a popup box appears on the screen:

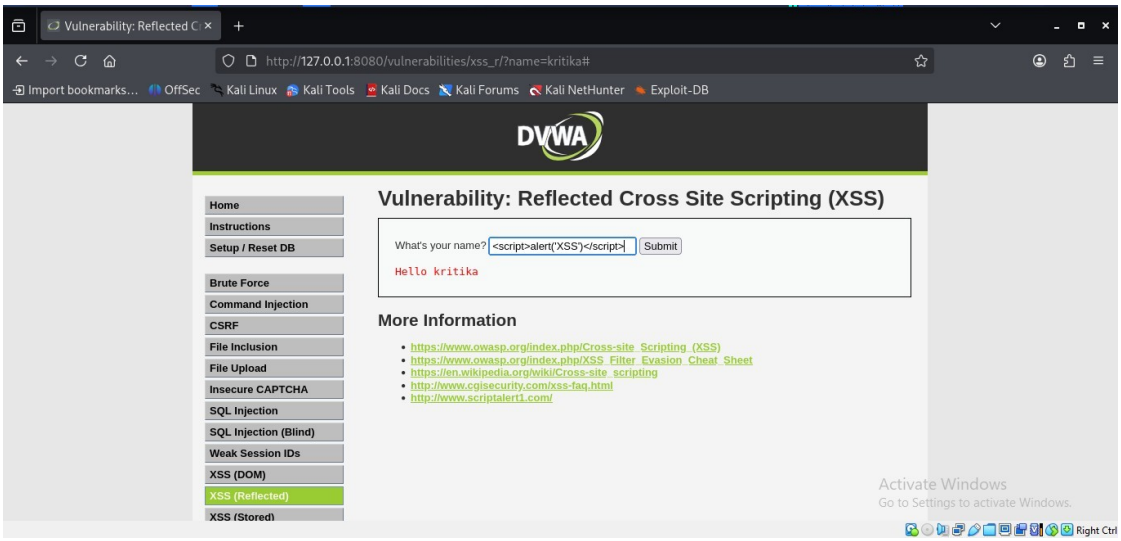


Fig 6.5 - Script typed into the input box.

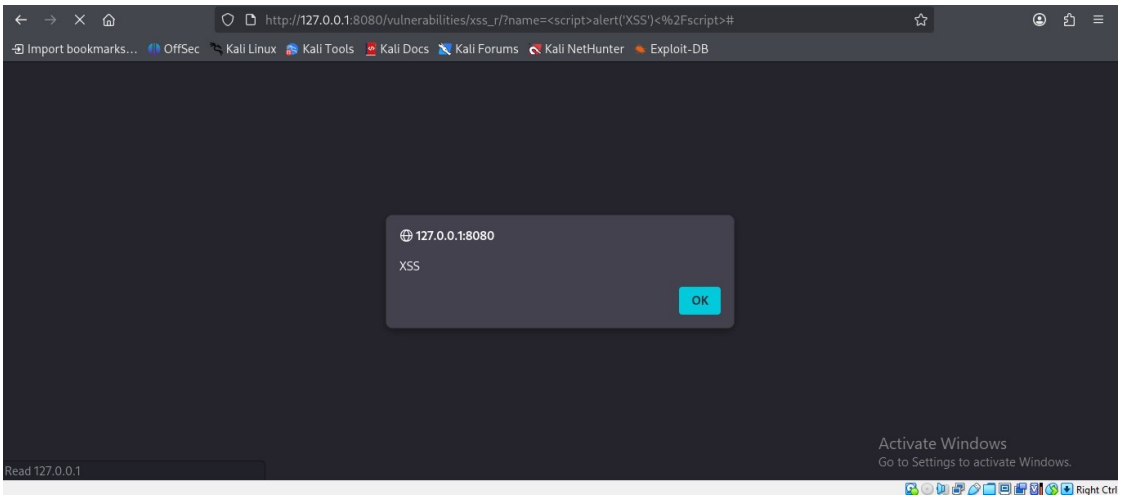


Fig 6.6 - Alert popup appears, meaning our script actually got executed.

Conclusion: This page is vulnerable to Reflected XSS because whatever we type in the box gets added back into the page without checking if it's safe or not.

6.3 Stored Cross-Site Scripting (XSS)

Tested on: DVWA → XSS (Stored) page (this is basically a guestbook where people can leave messages).

This time, instead of a normal message, I typed a script in the Message box and clicked Sign Guestbook:

```
<script>alert('Stored XSS')</script>
```

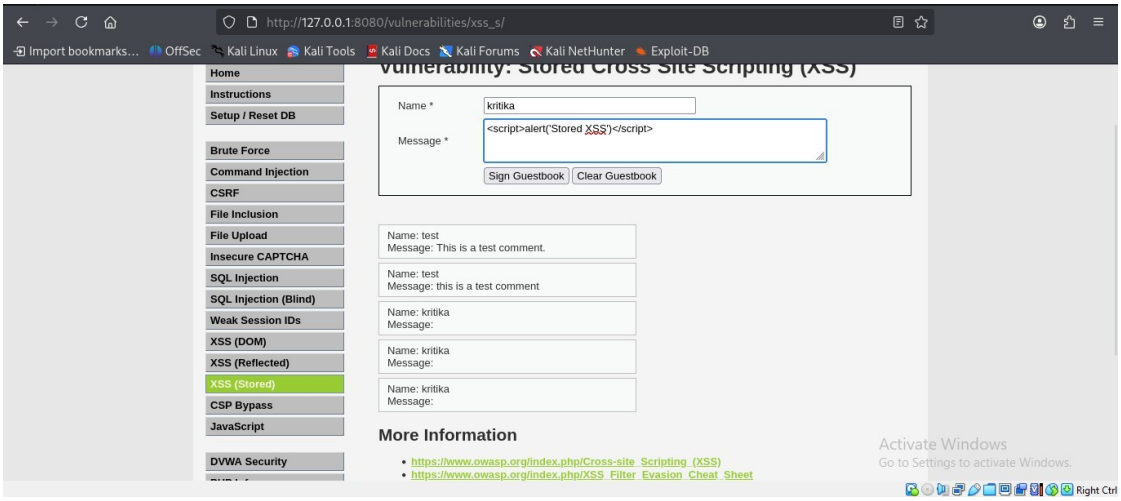


Fig 6.7 - Script entered in the guestbook Message field.

As soon as I submitted it, the popup appeared:

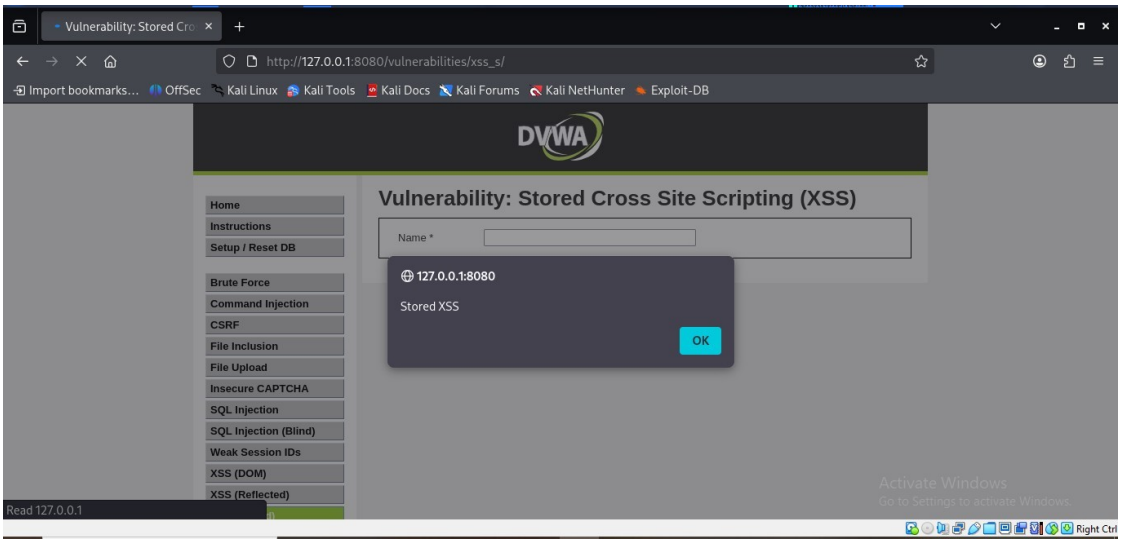


Fig 6.8 - Popup appears right after submitting the message.

What makes this different from reflected XSS is that the message actually gets saved on the server. So even after I reload the page or come back later, the same popup shows up again, because it's permanently stored in the database now:

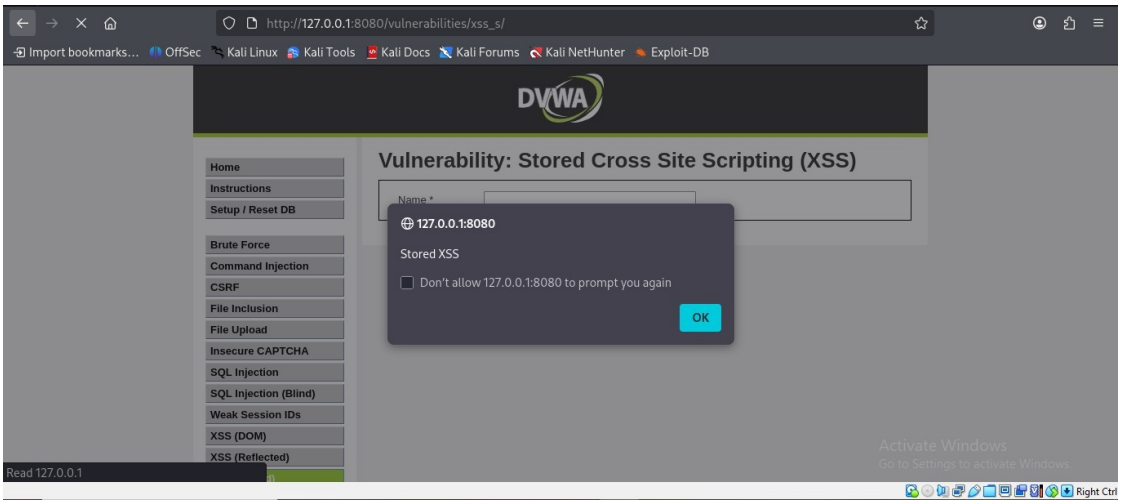


Fig 6.9 - Popup appears again even after reloading the page, proving it's saved on the server.

I also captured the POST request in Burp Suite while submitting this message:

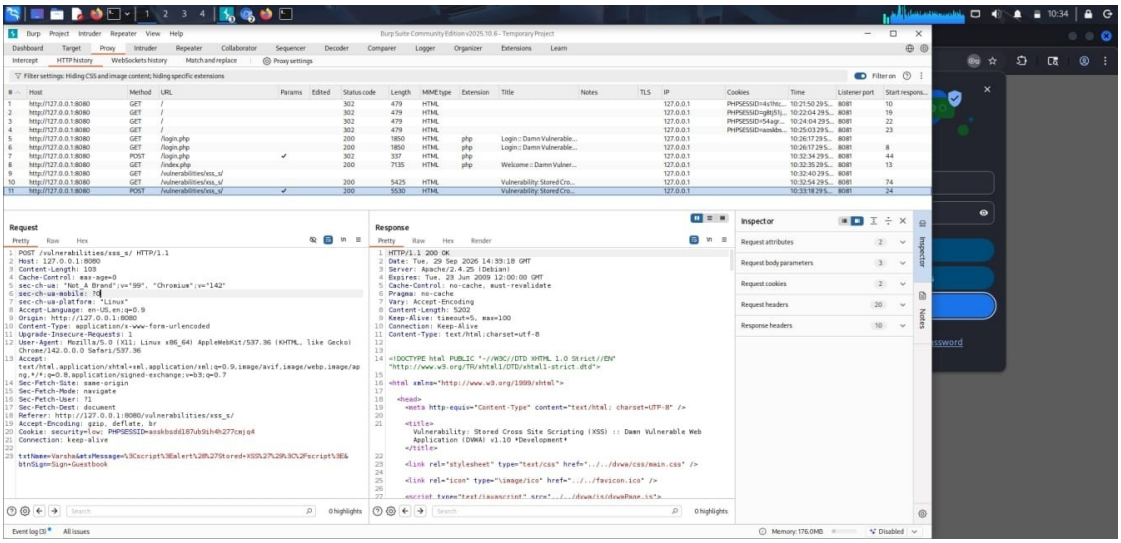
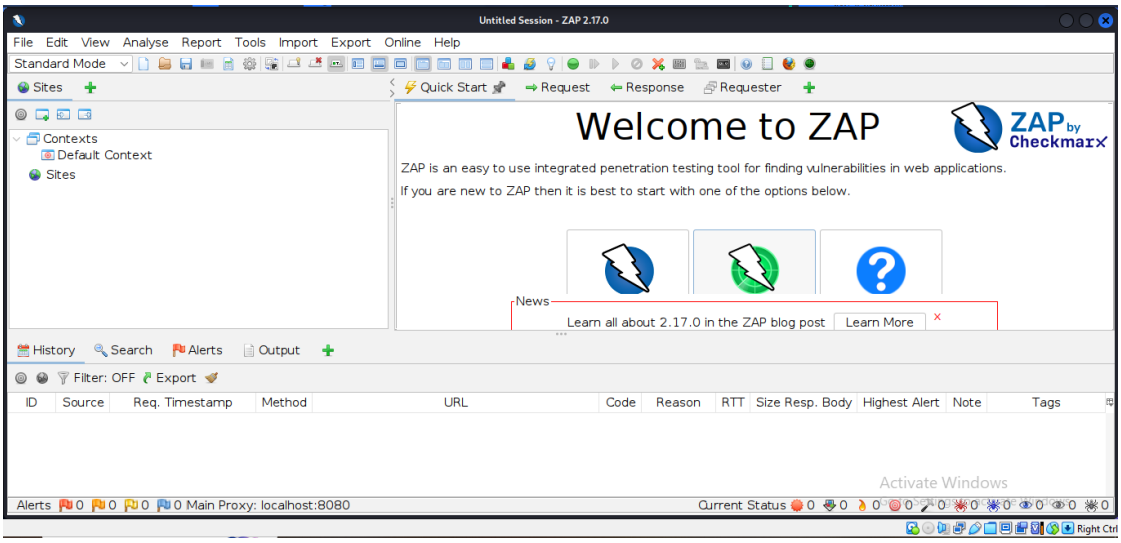


Fig 6.10 - Burp Suite capture of the Stored XSS request and response.

Conclusion: This is more dangerous than reflected XSS because the script is saved permanently and will run for every single person who opens that guestbook page, not just the person who entered it.

6.4 Automated Scanning with OWASP ZAP

Along with manually testing SQLi and XSS, I also ran an automated scan on DVWA using OWASP ZAP's Automated Scan option, by just giving it the DVWA URL (http://127.0.0.1:8080).


Untitled Session - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Welcome to ZAP
ZAP by Checkmarx
ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the options below.
News
Learn all about 2.17.0 in the ZAP blog post
Learn More
History
Search
Alerts
Output
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags
Activate Windows
Alerts 0 0 0 0 Main Proxy: localhost:8080
Current Status 0 0 0 0

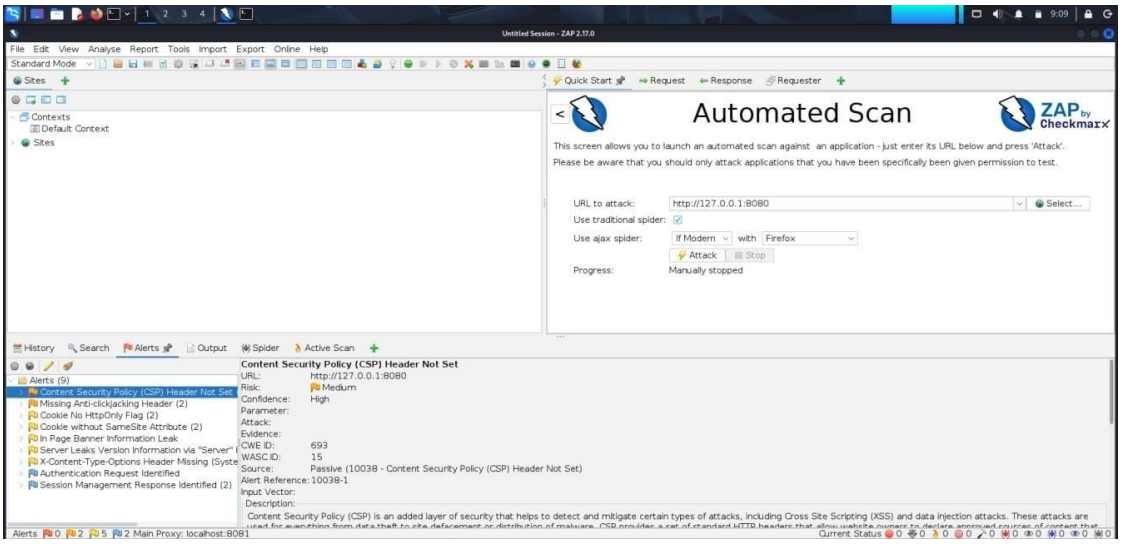


Fig 6.12 - ZAP alerts list (9 alerts found), with CSP header issue expanded.

Apart from SQLi and XSS, ZAP also pointed out some other smaller issues like missing security headers (CSP, clickjacking protection), cookies not having proper flags, and the server showing its version info, which an attacker could use to find known bugs for that exact version.

7. Findings & Risk Levels

Here is a summary of everything I found during this testing, along with how risky each one is:

Vulnerability	Location	Risk	Impact
SQL Injection	User ID field (sqli page)	High	Attacker can bypass authentication logic and dump entire database table contents (all user records) via a crafted boolean payload.
Reflected Cross-Site Scripting (XSS)	'name' GET parameter (xss_r page)	Medium	Script supplied in the URL is reflected back unsanitized and executes in the victim's browser — usable for session/cookie theft via a crafted link.
Stored Cross-Site Scripting (XSS)	Guestbook 'Message' field (xss_s page)	High	Malicious script is permanently stored in the database and executes for every user who views the guestbook — a persistent, self-propagating risk.
Content Security Policy (CSP) Header Not Set	Application-wide (all pages)	Medium	Absence of a CSP header removes a key browser-side control that would otherwise limit the impact of injected scripts.
Missing Anti-Clickjacking Header	Application-wide	Medium	Pages can be framed by a malicious site, enabling clickjacking attacks against logged-in users.
Cookie Without HttpOnly / SameSite Flags	PHPSESSID session cookie	Low	Session cookie is readable by client-side scripts and sent on cross-site requests, increasing session-hijacking risk.
Server Leaks	HTTP Server	Low	Disclosure of Apache/PHP version information helps an

Vulnerability	Location	Risk	Impact
Version Information	header		attacker select known exploits for that version.

8. Mitigation Methods

Based on what I found, here's how each of these issues can be fixed:

8.1 SQL Injection

- Use prepared statements / parameterized queries instead of directly putting user input into the SQL query.
- Validate user input on the server side (for example, only allow numbers in an ID field).
- Give the database user account only the permissions it actually needs, not full access.
- A Web Application Firewall (WAF) can also help block common SQLi patterns.

8.2 Reflected & Stored XSS

- Encode user input properly before showing it back on the page, so that it's treated as plain text and not as code.
- Don't allow tags like <script> in user input unless it's really needed.
- Use a Content Security Policy (CSP) header, which stops the browser from running unknown/unwanted scripts.
- Set the HttpOnly flag on cookies so scripts can't read them, even if XSS happens.

8.3 Broken Authentication (not manually tested — general OWASP guidance)

- Use strong password rules and lock the account after too many wrong login attempts.
- Generate a new session ID after login instead of reusing the old one.
- Use multi-factor authentication (MFA) wherever possible.

8.4 Security Misconfiguration

- Add missing security headers like Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options.
- Turn off detailed error messages and hide the server/PHP version in production.
- Keep the server, PHP, and all software updated with the latest patches.
- Run scans like OWASP ZAP regularly, not just once, to catch new issues early.

9. Conclusion

Doing this assignment helped me actually understand how SQL Injection and XSS work, instead of just reading about them. Using DVWA, I was able to see the difference between how an application behaves normally and how it behaves when a malicious payload is entered, for both SQL Injection and XSS (reflected and stored). I also got to practice using Burp Suite to look at raw requests/responses, and OWASP ZAP to run an automated scan and find other issues.

The biggest thing I learned is that almost all of these problems happen because the application trusts user input too much, without checking or cleaning it first. Simple fixes like using prepared statements, encoding output properly, and adding security headers can prevent most of these attacks. Overall, this exercise gave me a much better practical understanding of how web vulnerabilities actually work and why secure coding practices matter.

10. References

- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- OWASP Cross-Site Scripting (XSS) — https://owasp.org/www-community/attacks/xss/
- OWASP SQL Injection — https://owasp.org/www-community/attacks/SQL_Injection
- DVWA (Damn Vulnerable Web Application) — https://github.com/digininja/DVWA
- Burp Suite Documentation — https://portswigger.net/burp/documentation
- OWASP ZAP Documentation — https://www.zaproxy.org/docs/